SIMATS ENGINEERING

ASSESSMENT TOOL 1

Name: Navuluri Anjali
Reg No: 192324137
Marks: 92/100

COURSE NAME: INTERNET PROGRAMMING **COURSE CODE: CSA43**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
  <title>Company</title>
 </head>
 <body>
  <div>
   <h1>Welcome
   <p>About us
  </div>
</body> </html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I Navuluri Anjali (192324137) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Anjali

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns	23
Explanation	25%	Detailed, wellexplained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	23
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	18
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	14
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	9
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	5
	100%					92/100

1. Online Symposium Portal

1. Identify the HTML5 semantic elements missing in the structure.

HTML5 semantic elements clearly describe the purpose of different sections of a webpage. The given form contains input fields but does not include semantic elements to organize the webpage properly.
The following semantic elements can be added:

- <header> – Displays the title of the symposium portal.
- <nav> – Provides navigation links to different pages. • <main> – Contains the main content of the webpage.
- <section> – Groups related content, such as the registration form.
- <footer> – Displays contact details and copyright information.

These elements improve the structure, readability, accessibility, and maintainability of the webpage.

2. Modify the structure to make it standards-compliant.

The following HTML5 code uses semantic elements, proper labels, and suitable form attributes.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Online Symposium Registration</title>
</head>
<body>
   <header>
      <h1>Online Symposium Portal</h1>
   </header>
   <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
      <a href="/contact">Contact</a>
   </nav>
   <main>
      <section>
         <h2>Student Registration</h2>
         <form action="/register" method="POST">
            <label for="name">Student Name:</label>
<input type="text" id="name" name="name"
autocomplete="name" required>
            <br><br>
```

```
            <label for="email">Email Address:</label>
<input type="email" id="email" name="email"
autocomplete="email" required>
            <br><br>
```

Explanation: The modified code follows HTML5 standards by including a document type, language, character encoding, viewport settings, semantic elements, labels, and appropriate input attributes. The required attribute ensures that the student enters both fields before submitting the form.



Figure 1: Output of Online Symposium Registration Portal

3. Interpret the HTTP request generated during form submission.

When the student clicks the Submit button, the browser collects the form data and sends an HTTP POST request to the server.
For example, the request may look like this:
POST /register HTTP/1.1
Host: college.edu

Content-Type: application/x-www-form-urlencoded

name=Anjali&email=anjali%40gmail.com Explanation
of the request:

1. POST method: Sends the registration data to the server.
2. /register: Specifies the server endpoint that processes the registration.
3. Host header: Identifies the server receiving the request.
4. Content-Type: Indicates that the submitted data is encoded as URL-encoded form data.
5. Request body: Contains the student's name and email address.

The server receives the data, validates it, and may store the registration details in a database. Note: The exact request headers and protocol version may vary. If the portal uses HTTPS, the request is encrypted during transmission.

4. Analyze the HTTP response cycle from server to browser.

The HTTP response cycle describes how the server processes the student's request and sends a response to the browser.
Step 1: Student submits the form
The browser collects the name and email.
Step 2: Browser sends HTTP POST
The data is sent to the /register endpoint.
Step 3: Server processes the request
Validates the data and saves the registration if valid.
Step 4: Server sends an HTTP response
Returns a status code, headers, and optionally a response body.
Step 5: Browser displays the result
Shows confirmation or an appropriate error message.
Common HTTP response status codes include:

- 200 OK: The request was successful.
- 201 Created: The registration was successfully created.
- 400 Bad Request: The submitted data is invalid.
- 500 Internal Server Error: An unexpected server-side error occurred.

For example, after successful registration, the server may return a 201 Created response with a message such as “Registration successful.” The browser then displays the confirmation to the student.

5. Suggest improvements for usability and accessibility.

The following improvements can make the symposium portal easier and safer to use.

1. Add labels: Use the <label> element for every input field so users can understand what information is required.
2. Add validation: Use required and type="email" to check that the student enters the necessary information correctly.

3. Use semantic elements: Include <header>, <nav>, <main>, <section>, and <footer> to organize the webpage.
4. Provide clear feedback: Display success messages after registration and helpful error messages when submission fails.
5. Support keyboard navigation: Ensure that all fields and buttons can be accessed using the keyboard.
6. Improve visual design: Use readable fonts, suitable spacing, clear button text, and sufficient color contrast.
7. Make the page responsive: Use the viewport meta tag and responsive CSS so the portal works on mobile phones, tablets, and computers.
8. Protect student information: Use HTTPS, validate data on the server, and handle personal information securely.
9. Prevent duplicate registrations: Check whether the student has already registered before creating a new record.
10. Support assistive technologies: Use meaningful headings, accessible error messages, and appropriate ARIA attributes when necessary.

Conclusion

The online symposium portal can be improved by using HTML5 semantic elements, a standards-compliant registration form, and proper HTTP request and response handling. Adding validation, accessibility features, responsive design, and secure data processing will make the portal more reliable, user-friendly, and accessible to all students.

2. Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags.

The given HTML code contains several errors that may affect the webpage's structure.

- Missing <!DOCTYPE html>: The document does not declare HTML5, which may cause browsers to use quirks mode.
- Missing <html lang="en"> attribute: The language of the webpage is not specified.
- Missing <meta charset="UTF-8">: Character encoding is not declared.
- Missing closing </h1> tag: The heading starts with <h1> but is not properly closed.
- Missing closing tag: The paragraph starts with but is not properly closed.
- Missing semantic elements: Elements such as <header> and <main> are not used to organize the content.

These errors should be corrected to make the webpage valid and consistent across browsers.

2. Analyze how different browsers may interpret this structure.

Different browsers may handle invalid HTML by applying their own error-recovery rules.

1. Automatic tag correction: Browsers may automatically close some unclosed elements while parsing the HTML.
2. Different DOM structures: Browsers may construct or repair the document structure in ways that affect how scripts and styles interact with the page.
3. Layout differences: Headings and paragraphs have default styles, so incorrect markup may produce unexpected spacing or formatting.
4. Compatibility problems: The absence of a proper document type may cause quirks mode, where some CSS layout rules behave differently from standards mode.
5. Maintenance difficulties: Invalid markup makes debugging and future modifications more difficult.

For example, one browser may display the heading and paragraph in a way that appears correct, but the page can still contain invalid HTML and behave unexpectedly when CSS or JavaScript is added.

3. Identify missing semantic elements.

HTML5 semantic elements describe the purpose of different parts of a webpage.

- <header>: Contains the company name, logo, or introductory heading.
- <nav>: Contains navigation links to pages such as Home, About, and Contact.
- <main>: Contains the main content of the webpage.

- <section>: Groups related content, such as the About Us section.
- <footer>: Contains copyright information and contact details.

Using these elements improves accessibility, readability, and the organization of the webpage.

4. Provide a corrected W3C-compliant HTML structure.

The following code corrects the missing tags and uses appropriate HTML5 semantic elements.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>
<body>
   <header>
      <h1>Welcome to Our Company</h1>
   </header>
   <nav>
      <a href="/">Home</a>
      <a href="/about">About Us</a>
     <a href="/contact">Contact</a>
   </nav>
   <main>
      <section>
         <h2>About Us</h2>
         <p>We provide quality services to our customers.</p>
      </section>
   </main>
   <footer>
      <p>Copyright 2026 Company. All rights reserved.</p>
   </footer>
</body>
</html>
```

Explanation:

- <!DOCTYPE html> declares the HTML5 document type.
- <html lang="en"> specifies the language of the page.
- <meta charset="UTF-8"> defines the character encoding.
- <meta name="viewport"> helps the page display properly on mobile devices.
- All heading and paragraph tags are properly opened and closed.

- Semantic elements organize the webpage into meaningful sections.

This structure follows standard HTML syntax and can be checked using the W3C Markup Validation Service.



Figure 2: Output of Cross-Browser Compatible Company Website

5. Suggest techniques to ensure cross-browser compatibility.

The following techniques help a website display consistently across browsers such as Chrome, Firefox, Edge, and Safari.

1. Use valid HTML: Write properly nested elements and close all required tags.
2. Validate the code: Use the W3C Markup Validation Service to identify HTML errors.
3. Use CSS resets or normalization: Reduce differences in default browser styles.
4. Declare the HTML5 doctype: Include <!DOCTYPE html> to enable standards mode.
5. Use standard CSS properties: Avoid relying on browser-specific behavior; use vendor prefixes only when necessary.
6. Test across browsers: Check the website in different browsers and on different devices.

7. Use responsive design: Apply CSS media queries and flexible layouts to support different screen sizes.
8. Avoid deprecated elements: Use modern HTML5 elements instead of outdated tags.
9. Use feature detection: Check whether a browser supports a feature before depending on it.
10. Keep browsers updated: Modern browser versions generally provide better standards support and security.

Conclusion

Cross-browser compatibility ensures that a website works correctly across different web browsers. By correcting missing tags, using semantic HTML5 elements, validating the code, and testing across browsers, developers can create a consistent, accessible, and standardscompliant website.

3. Form Submission Failure

1. Identify issues in the form configuration.

The given HTML form contains several issues that prevent it from submitting data correctly.

- Incorrect button type: The button uses type="button", which does not submit the form automatically.
- Incorrect HTTP method: The form uses GET to send a username and password. This is insecure because form values are added to the URL.
- Missing required attributes: The username and password fields can be left empty.
- Missing labels: The input fields do not have labels to help users understand what information to enter.
- Missing validation: The form does not enforce basic input requirements. These issues should be corrected to ensure proper form submission and protect user information.

2. Analyze why the HTTP request is not triggered.

The main reason is the incorrect button type.
The given code contains:

<button type="button">Login</button>

The type="button" attribute creates a normal button. Clicking it does not submit the form unless JavaScript is used to perform that action.

Therefore, the browser does not automatically send an HTTP request to the /submit endpoint.

Solution: Change the button type to submit.

<button type="submit">Login</button>

When the user clicks the button, the browser submits the form data to the server, provided the form is valid and no other script or browser condition prevents submission.

3. Interpret the role of the GET method in this scenario.

The GET method requests information from a server. When used in a form, the browser appends the form data to the URL as query parameters.

For example:
GET /submit?username=Anjali&password=12345 HTTP/1.1
Host: example.com

The data appears in the URL after the question mark (?). Disadvantages of GET for login forms:

1. Passwords may appear in browser history and server logs.
2. URLs may be stored or shared accidentally.
3. The amount of data that can be sent is limited by practical URL-length restrictions.
4. GET requests are intended primarily for retrieving data, not performing login submissions that change server-side state.

Therefore, GET should not be used to transmit passwords. A login form should generally use the POST method over HTTPS.

5. **Propose a corrected implementation.**

The following HTML code fixes the button issue, uses POST, and includes labels and basic validation.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Form</title>
</head>
<body>
    <h1>Login</h1>
    <form action="/submit" method="POST">
        <label for="username">Username:</label>
<input type="text" id="username" name="username"
required autocomplete="username">
        <br><br>
        <label for="password">Password:</label>
        <input type="password" id="password" name="password"
required autocomplete="current-password">
        <br><br>
        <button type="submit">Login</button>
    </form>
</body>
</html>
```

Explanation:

- action="/submit" specifies the server endpoint that processes the form.
- method="POST" sends the form data in the HTTP request body rather than in the URL.
- type="submit" triggers form submission.
- required prevents submission when either field is empty.
- <label> elements improve usability and accessibility.

- autocomplete helps browsers identify the username and password fields.

The server must also validate the credentials and return an appropriate response.



Figure 3: Output of Corrected Login Form

6. Suggest improvements for secure data transmission.

The following improvements help protect login information.

1. Use HTTPS: Encrypts data during transmission between the browser and server.
2. Use POST for login forms: Keeps credentials out of the URL, although POST alone does not encrypt data.
3. Validate data on the server: Never rely only on browser-side validation.
4. Hash passwords securely: Store passwords using a strong password-hashing algorithm such as Argon2id or bcrypt, rather than storing plain-text passwords.
5. Use secure session management: Protect session cookies with Secure, HttpOnly, and appropriate SameSite attributes.
6. Prevent brute-force attacks: Apply rate limiting and other appropriate login protections.

7. Avoid logging sensitive information: Do not store passwords in application logs or error messages.
8. Display safe error messages: Avoid revealing whether a particular username or password is correct unnecessarily.

Conclusion

The primary cause of the form submission failure is the use of type="button" instead of type="submit". Changing the button type and using the POST method enables proper form submission. Using HTTPS, server-side validation, secure password storage, and safe session management makes the login process more secure and reliable.